Solution: Number of Good Pairs

Problem Statement

Given an array of integers `nums`, return the number of **good pairs** in it.

A pair `(i, j)` is called **good** if `nums[i] == nums[j]` and `i < j`.

Example 1:

```
Input: nums = [1,2,3,1,1,3]
Output: 4
Explanation: There are 4 good pairs, here are the indices: (0,3), (0,4), (3,4), (2,5).
```

Example 2:

```
Input: nums = [1,1,1,1]
Output: 6
Explanation: Each pair in the array is a 'good pair'.
```

Example 3:

```
Input: words = nums = [1,2,3]
Output: 0
Explanation: No number is repeating.
```

Solution

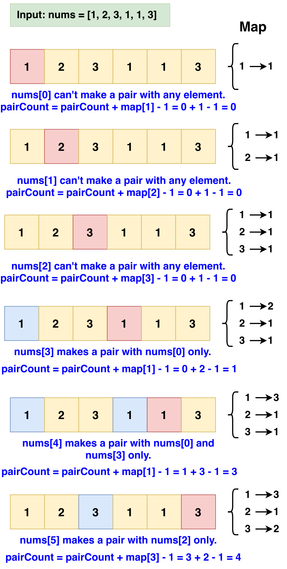
We can use a `HashMap` to store the frequency of each number in the input array `nums`.

While iterating through the input array, for each number `n` in the array, we will increment the count of `n` in the `HashMap`.

Whenever we find a new occurrence of a number, we have found a new pair.

Every new occurrence of a number can be paired with every previous occurrence of the same number. This means if a number has already appeared `p` times, we will have `p-1` new pairs.

Hence, whenever we find a new occurrence of a number (that is, its count is more than `1`), we will add `p-1` to `pairCount`, which keeps track of the total number of good pairs.



Code

Here is the code for this algorithm:

Java

```java
import java.util.*;

class Solution {
  public int numGoodPairs(int[] nums) {
    int pairCount = 0;
    Map<Integer, Integer> map = new HashMap<>();
    for (int n : nums) {
      map.put(n, map.getOrDefault(n, 0) + 1); // increment the count of 'n' in the map
      // every new occurrence of a number can be paired with every previous occurrence
      // so if a number has already appeared 'p' times, we will have 'p-1' new pairs
      pairCount += map.get(n) - 1;
    }
    return pairCount;
  }

  public static void main(String[] args) {
    Solution sol = new Solution();

    int[] nums1 = { 1, 2, 3, 1, 1, 3 };
    int result1 = sol.numGoodPairs(nums1);
    System.out.println("Result 1: " + result1 + " (Expected: 4)");
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the algorithm is $O(n)$, where n is the length of the input array `nums`. The reason for this is that the algorithm only needs to loop through each element of the array once, and the operations performed inside the loop (putting elements into the map, getting their values, and incrementing pairCount) all have constant time complexity.

Space Complexity

The space complexity of the algorithm is $O(n)$, where n is the length of the input array `nums`. This is because the algorithm uses a HashMap to store the frequency of each number in the input array, which can take up to $O(n)$ space in the worst case scenario if all the elements in the array are unique.